My Project

Generated by Doxygen 1.16.1

1 Aprašymas 1
1.1 Release v.pradinė . . . 1
1.2 Release v0.1 . . . 1
1.3 Release v0.2 . . . 1
1.4 Release v0.3 . . . 1
1.5 Release v0.4 . . . 2
1.6 Release v1.0-initial . . . 2
1.7 Testavimo sistemos parametrai: . . . 2
1.8 Laiko matavimų lentelė . . . 2
1.9 Pakartotinų matavimų lentelė . . . 2
1.10 Laiko matavimų vidurkis . . . 3
1.11 Release v1.0 . . . 3
1.12 1 strategija (pateikiamas kelių testų laikų vidurkis) . . . 3
1.13 2 strategija (pateikiamas kelių testų laikų vidurkis) . . . 3
1.14 3 strategija (pateikiamas kelių testų laikų vidurkis) . . . 4
1.15 Versija 1.1. . . . 4
1.16 Laiko matavimų lentelė (pateikiamas kelių testų laikų vidurkis) . . . 5
1.17 exe failų dydžių lentelė . . . 6
1.18 Versija 1.2. . . . 6
1.19 Student klasės struktūra . . . 6
1.20 Išvesties ir įvesties operatorių perdengimas . . . 7
1.21 Versija 1.5. . . . 7
1.22 Versija 2.0. . . . 8
1.23 Versija 3.0. . . . 8
1.24 Naudojimosi instrukcija . . . 9

2 Directory Hierarchy 11
2.1 Directories . . . 11

3 Hierarchical Index 13
3.1 Class Hierarchy . . . 13

4 Class Index 15
4.1 Class List . . . 15

5 File Index 17
5.1 File List . . . 17

6 Directory Documentation 19
6.1 include Directory Reference . . . 19
6.2 src Directory Reference . . . 19

7 Class Documentation 21
7.1 Person Class Reference . . . 21

7.1.1 Constructor & Destructor Documentation . . . 21
7.1.1.1 Person() [1/3] . . . 21
7.1.1.2 ~Person() . . . 22
7.1.1.3 Person() [2/3] . . . 22
7.1.1.4 Person() [3/3] . . . 22
7.1.2 Member Function Documentation . . . 22
7.1.2.1 getName() . . . 22
7.1.2.2 getSurname() . . . 22
7.1.2.3 operator=() [1/2] . . . 22
7.1.2.4 operator=() [2/2] . . . 22
7.1.2.5 print() . . . 22
7.1.3 Member Data Documentation . . . 23
7.1.3.1 name_ . . . 23
7.1.3.2 surname_ . . . 23
7.2 Student Class Reference . . . 23
7.2.1 Constructor & Destructor Documentation . . . 24
7.2.1.1 Student() [1/3] . . . 24
7.2.1.2 ~Student() . . . 24
7.2.1.3 Student() [2/3] . . . 24
7.2.1.4 Student() [3/3] . . . 24
7.2.2 Member Function Documentation . . . 25
7.2.2.1 addNd() . . . 25
7.2.2.2 getEgz() . . . 25
7.2.2.3 getFinalAvg() . . . 25
7.2.2.4 getFinalMed() . . . 25
7.2.2.5 getNd() . . . 25
7.2.2.6 operator=() [1/2] . . . 25
7.2.2.7 operator=() [2/2] . . . 25
7.2.2.8 print() . . . 25
7.2.2.9 setEgz() . . . 25
7.2.2.10 setNd() . . . 26
7.2.3 Friends And Related Symbol Documentation . . . 26
7.2.3.1 operator<< . . . 26
7.2.3.2 operator>> . . . 26
7.3 Vector< T > Class Template Reference . . . 26
7.3.1 Constructor & Destructor Documentation . . . 27
7.3.1.1 Vector() [1/3] . . . 27
7.3.1.2 ~Vector() . . . 27
7.3.1.3 Vector() [2/3] . . . 27
7.3.1.4 Vector() [3/3] . . . 27
7.3.2 Member Function Documentation . . . 27
7.3.2.1 assign() . . . 27

7.3.2.2 at() [1/2] . . . 28
7.3.2.3 at() [2/2] . . . 28
7.3.2.4 back() [1/2] . . . 28
7.3.2.5 back() [2/2] . . . 28
7.3.2.6 begin() [1/2] . . . 28
7.3.2.7 begin() [2/2] . . . 28
7.3.2.8 capacity() . . . 28
7.3.2.9 clear() . . . 28
7.3.2.10 empty() . . . 29
7.3.2.11 end() [1/2] . . . 29
7.3.2.12 end() [2/2] . . . 29
7.3.2.13 erase() [1/2] . . . 29
7.3.2.14 erase() [2/2] . . . 29
7.3.2.15 front() [1/2] . . . 29
7.3.2.16 front() [2/2] . . . 29
7.3.2.17 insert() . . . 29
7.3.2.18 operator=() [1/2] . . . 30
7.3.2.19 operator=() [2/2] . . . 30
7.3.2.20 operator[]() [1/2] . . . 30
7.3.2.21 operator[]() [2/2] . . . 30
7.3.2.22 pop_back() . . . 30
7.3.2.23 push_back() . . . 30
7.3.2.24 reserve() . . . 30
7.3.2.25 resize() . . . 30
7.3.2.26 size() . . . 30

8 File Documentation **31**
8.1 include/calculations.h File Reference . . . 31
8.1.1 Function Documentation . . . 31
8.1.1.1 avg() . . . 31
8.1.1.2 compareByAvg() . . . 31
8.1.1.3 compareByMed() . . . 32
8.1.1.4 compareByName() . . . 32
8.1.1.5 compareBySurname() . . . 32
8.1.1.6 GenerateName() . . . 32
8.1.1.7 med() . . . 32
8.2 calculations.h . . . 32
8.3 include/fileHandling.h File Reference . . . 33
8.3.1 Function Documentation . . . 33
8.3.1.1 createFile() . . . 33
8.3.1.2 readFile() . . . 33
8.4 fileHandling.h . . . 33

8.5 include/inputValid.h File Reference . . . 34
8.5.1 Function Documentation . . . 34
8.5.1.1 inputInt() . . . 34
8.5.1.2 inputLetter() . . . 34
8.5.1.3 inputR() . . . 34
8.5.1.4 inputWord() . . . 34
8.6 inputValid.h . . . 34
8.7 include/meniu.h File Reference . . . 35
8.7.1 Function Documentation . . . 35
8.7.1.1 generateFile() . . . 35
8.7.1.2 generateGrades() . . . 35
8.7.1.3 generateNamesGrades() . . . 35
8.7.1.4 handleOutput() . . . 36
8.7.1.5 manualInput() . . . 36
8.7.1.6 readFile() . . . 36
8.7.1.7 scanFile() . . . 36
8.7.1.8 showMeniu() . . . 36
8.7.1.9 sortingStudents() . . . 36
8.8 meniu.h . . . 37
8.9 include/output.h File Reference . . . 37
8.9.1 Function Documentation . . . 37
8.9.1.1 handleOutput() . . . 37
8.9.1.2 outputToFileSorting() . . . 38
8.9.1.3 printResults() . . . 38
8.10 output.h . . . 38
8.11 include/person.h File Reference . . . 38
8.12 person.h . . . 39
8.13 include/student.h File Reference . . . 39
8.14 student.h . . . 39
8.15 include/vector.h File Reference . . . 40
8.16 vector.h . . . 40
8.17 README.md File Reference . . . 44
8.18 src/calculations.cpp File Reference . . . 44
8.18.1 Function Documentation . . . 44
8.18.1.1 compareByAvg() . . . 44
8.18.1.2 compareByMed() . . . 44
8.18.1.3 compareByName() . . . 44
8.18.1.4 compareBySurname() . . . 44
8.18.1.5 GenerateName() . . . 45
8.19 src/fileHandling.cpp File Reference . . . 45
8.19.1 Function Documentation . . . 45
8.19.1.1 createFile() . . . 45

8.19.1.2 createFile< std::deque< Student > >() . . . 45
8.19.1.3 createFile< std::list< Student > >() . . . 46
8.19.1.4 createFile< std::vector< Student > >() . . . 46
8.19.1.5 readFile() . . . 46
8.19.1.6 readFile< std::deque< Student > >() . . . 46
8.19.1.7 readFile< std::list< Student > >() . . . 46
8.19.1.8 readFile< std::vector< Student > >() . . . 46
8.19.1.9 readFile< Vector< Student > >() . . . 46
8.20 src/inputValid.cpp File Reference . . . 47
8.20.1 Function Documentation . . . 47
8.20.1.1 inputInt() . . . 47
8.20.1.2 inputLetter() . . . 47
8.20.1.3 inputR() . . . 47
8.20.1.4 inputWord() . . . 47
8.21 src/main.cpp File Reference . . . 47
8.21.1 Function Documentation . . . 48
8.21.1.1 main() . . . 48
8.22 src/meniu.cpp File Reference . . . 48
8.22.1 Function Documentation . . . 49
8.22.1.1 generateFile() . . . 49
8.22.1.2 generateGrades() . . . 49
8.22.1.3 generateGrades< std::deque< Student > >() . . . 49
8.22.1.4 generateGrades< std::list< Student > >() . . . 49
8.22.1.5 generateGrades< std::vector< Student > >() . . . 49
8.22.1.6 generateGrades< Vector< Student > >() . . . 50
8.22.1.7 generateNamesGrades() . . . 50
8.22.1.8 generateNamesGrades< std::deque< Student > >() . . . 50
8.22.1.9 generateNamesGrades< std::list< Student > >() . . . 50
8.22.1.10 generateNamesGrades< std::vector< Student > >() . . . 50
8.22.1.11 generateNamesGrades< Vector< Student > >() . . . 50
8.22.1.12 handleOutput< Vector< Student > >() . . . 50
8.22.1.13 manualInput() . . . 50
8.22.1.14 manualInput< std::deque< Student > >() . . . 51
8.22.1.15 manualInput< std::list< Student > >() . . . 51
8.22.1.16 manualInput< std::vector< Student > >() . . . 51
8.22.1.17 manualInput< Vector< Student > >() . . . 51
8.22.1.18 readFile< Vector< Student > >() . . . 51
8.22.1.19 scanFile() . . . 51
8.22.1.20 scanFile< std::deque< Student > >() . . . 51
8.22.1.21 scanFile< std::list< Student > >() . . . 51
8.22.1.22 scanFile< std::vector< Student > >() . . . 52
8.22.1.23 scanFile< Vector< Student > >() . . . 52

8.22.1.24 showMeniu() 52
8.22.1.25 sortingStudents() 52
8.22.1.26 sortingStudents< std::deque< Student > >() 52
8.22.1.27 sortingStudents< std::list< Student > >() 52
8.22.1.28 sortingStudents< std::vector< Student > >() 52
8.22.1.29 sortingStudents< Vector< Student > >() 53
8.23 src/output.cpp File Reference 53
8.23.1 Function Documentation 54
8.23.1.1 handleOutput() 54
8.23.1.2 handleOutput< std::deque< Student > >() 54
8.23.1.3 handleOutput< std::list< Student > >() 54
8.23.1.4 handleOutput< std::vector< Student > >() 54
8.23.1.5 handleOutput< Vector< Student > >() 54
8.23.1.6 outputToFileSorting() 54
8.23.1.7 outputToFileSorting< std::deque< Student > >() 54
8.23.1.8 outputToFileSorting< std::list< Student > >() 54
8.23.1.9 outputToFileSorting< std::vector< Student > >() 55
8.23.1.10 outputToFileSorting< Vector< Student > >() 55
8.23.1.11 printResults() 55
8.23.1.12 printResults< std::deque< Student > >() 55
8.23.1.13 printResults< std::list< Student > >() 55
8.23.1.14 printResults< std::vector< Student > >() 55
8.23.1.15 printResults< Vector< Student > >() 55
8.23.1.16 sortContainer() [1/2] 56
8.23.1.17 sortContainer() [2/2] 56
8.24 src/person.cpp File Reference 56
8.25 src/student.cpp File Reference 56
8.25.1 Function Documentation 56
8.25.1.1 operator<<() 56
8.25.1.2 operator>>() 56

Index **57**

Chapter 1

Aprašymas

Ši programa skaičiuoja studentų galutinius pažymius pagal suvestus ranka arba sugeneruotus namų darbų pažymius bei egzamino įvertinimus.

1.1 Release v.pradinė

Sukurtas pagrindinis programos funkcionalumas: programa leidžia įvesti studentų vardus, pavardes, namų darbų rezultatus ir egzamino balą. Galutinis balas skaičiuojamas pagal namų darbų vidurkį arba medianą (vartotojas gali pasirinkti). Rezultatai pateikiami dviejų skaičių po kablelio tikslumu. Vieno studento duomenys saugomi struktūroje.

1.2 Release v0.1

Programa papildyta ir veikia su nežinomu studentų ir namų darbų skaičiumi prieš įvedimą. Yra dvi versijos: viena realizuota su C masyvais, kita - su std::vector. Pridėta galimybė atsitiktinai generuoti namų darbų ir egzamino įvertinimus. Sukurtas meniu skirtingiems programos vykdymo variantams: galima įvesti viską ranka, generuoti tik pažymius, generuoti studentų vardus, pavardes ir pažymius, baigti darbą.

1.3 Release v0.2

Programa papildyta, kad vartotojas galėtų nuskaityti studentų duomenis iš failo (tik std::vector versija). Programa leidžia pasirinkti duomenų rūšiavimo būdą: pagal vardą, pavardę arba galutinį pažymį (vidurkį arba medianą). Išvedimas galimas tiek terminale, tiek faile, išvestis išlygiuota.

1.4 Release v0.3

Funkcijų deklaracijos ir structūra perkelti į atskirus antraštinius (∗.h) failus. Programa padalinta į kelis ∗.cpp failus pagal funkcionalumą, kad būtų lengviau peržiūrėti ir pildyti kodą. Pridėtas klaidų gaudymo metodas (Eception Handling): gaudomos failų atidarymo klaidos, tikrinama įvestis, kad vartotojas negalėtų įvesti neleistinų reikšmių, visi pranešimai išvedami lietuvių kalba.

1.5 Release v0.4

Sukurta funkcija, kuri generuoja studentų failus pagal nurodytą įrašų skaičių. Vardai ir pavardės generuojami šabloniškai. Studentai su galutiniu balu $<$ 5.0 perkeliami į failą „vargšiukai“, studentai su galutiniu balu $>=$ 5.0 perkeliami į failą „kietiakai“. Buvo atlikti du tyrimai: buvo matuojama, kiek užtrunka sugeneruoti failą su nurodytu įrašų skaičiumi bei matuojamas duomenų apdorojimos laikas: failo nuskaitymas, studentų rūšiavimas į dvi grupes, išvedimas į failus, bendras programos vykdymo laikas. Tyrimų rezultatai užfiksuoti ir pateikti lentelėse README.md faile v0.4 šakoje su komentarais.

1.6 Release v1.0-initial

Po v0.4 versijos programos pritaikymo naudojimui su 3 skirtingų tipų konteineriais (vector, list, deque), buvo atliktas papildomas programos veikimo spartos tyrimas, kuriuo buvo siekiama įvertinti, kaip keičiasi programos našumas naudojant skirtingus C++ standartinės bibliotekos konteinerius. Kadangi failų kūrimo bei rezultatų išvedimo į failus sparta nepriklauso nuo naudojamo kontenerio tipo, šio tyrimo metu buvo matuojamas tik šių dalių įvykdymo sparta: duomenų nuskaitymas iš failo į pasirinktą konteinerį, studentų rūšiavimas didėjimo tvarka konteneryje, studentų skirstymas į dvi grupes. Tyrimo patikimumui visi testai buvo atliekami su tais pačiais failais 5 skirtingų dydžių: 1000, 10000, 100000, 1000000 ir 10000000 įrašų apimties.

Lentelėse pateikiamas bendras šių trijų operacijų vykdymo laikas.

1.7 Testavimo sistemos parametrai:

- CPU: AMD Ryzen 7 7730U with Radeon Graphics;
- RAM: 16 GB DDR4;
- Diskas: SSD 512 GB.

1.8 Laiko matavimų lentelė

Failo dydis	Programa su vector (s)	Programa su list (s)	Programa su deque (s)
1 000	0.0133683	0.0245043	0.013581
10 000	0.1332899	0.2441353	0.1357364
100 000	1.316732	2.518383	1.357291
1 000 000	12.47667	26.27343	18.68001
10 000 000	120.9164	263.7273	181.5412

1.9 Pakartotinų matavimų lentelė

Failo dydis	Programa su vector (s)	Programa su list (s)	Programa su deque (s)
1 000	0.0133868	0.0225857	0.013564
10 000	0.1343509	0.2437381	0.1339266

Failo dydis	Programa su vector (s)	Programa su list (s)	Programa su deque (s)
100 000	1.314891	2.419675	1.338495
1 000 000	12.59094	26.27369	18.69661
10 000 000	121.0477	263.9848	182.5063

1.10 Laiko matavimų vidurkis

Failo dydis	Programa su vector (s)	Programa su list (s)	Programa su deque (s)
1 000	0.01337755	0.024545	0.0135725
10 000	0.1338204	0.2439367	0.1348315
100 000	1.3158115	2.5160365	1.347893
1 000 000	12.533805	26.27356	18.68831
10 000 000	120.98205	263.85605	181.42375

Pagal laiko matavimų rezultatus matosi, jog sparčiausiai programa veikė su std::vector tipo konteineriais, o lėčiausiai su std::list, nors teoriškai sparčiausias turėjo būti std::vector, o lėčiausias std::list. Taip nutinka dėl konteinerių skirtingos atminties panaudojimo. std::vector naudoja vientisą atmintį, std::deque - blokinę atmintį, o std::list elementai pasiekiami per rodykles.

1.11 Release v1.0

Po pradinio relese išleidimo buvo padarytas programos optimizavimas 3 skirtingomis strategijomis su fiksuotu konteneriu - std::vector. Lentelėje pateikiami matavimų vidurkiai.

1.12 1 strategija (pateikiamas kelių testų laikų vidurkis)

Failo dydis	std::vector	std::list	std::deque
1 000	0.0130354	0.0223829	0.0139401
10 000	0.1240486	0.2324099	0.1570868
100 000	1.28094	2.332449	1.538205
1 000 000	12.72145	26.55926	18.5782
10 000 000	120.6255	261.365	180.152

std::vector yra greičiausias, nes naudoja vientisą atmintį ir pasižymi geriausia cache lokalizacija, std::deque - šiek tiek lėtesnis dėl segmentuotos atminties, std::list - lėčiausias, nes elementai saugomi per rodykles, o tai blogina cache panaudojimą.

1.13 2 strategija (pateikiamas kelių testų laikų vidurkis)

Failo dydis	std::vector	std::list	std::deque
1 000	0.0371157	0.0032958	0.0209625

Failo dydis	std::vector	std::list	std::deque
10 000	3.45011	0.0320269	2.27534
100 000	385.784	0.344122	174.784
1 000 000	> 1 h (nebaigta)	3.73521	> 1 h (nebaigta)
10 000 000	> 1 h (nebaigta)	32.9376	> 1 h (nebaigta)

Testavimas su std:: vector ir std::deque dideliems failams (1 000 000 ir 10 000 000 įrašų) nebuvo pilnai užbaigtas dėl labai ilgo vykdymo laiko (virš 1 valandos). Tai rodo, kad ši strategija su šiuo konteineriu yra neefektyvi dideliems duomenų kiekiams.

Galima pastebėti, kad su std::list 2 strategija veikia greičiausiai, o pats lečiausias yra std::vector. std::list konteineryje elementų šalinimas vykdomas O(1) sudėtingumu, nes pakanka pakeisti rodykles į gretimus elementus, nereikia perkelti likusių duomenų. Tuo tarpu std::vector konteineryje erase operacija yra O(n), nes po kiekvieno pašalinimo visi sekantys elementai turi būti perstumti atmintyje. Kadangi šioje strategijoje šalinimo operacija atliekama daug kartų, bendras veikimo laikas stipriai išauga, ypač dirbant su dideliais duomenų kiekiais. std::deque konteineris šiuo atveju pasirodo geriau nei std::vector, tačiau vis tiek ženkliai atsilieka nuo std::list. Taip yra todėl, kad skirtingai nei std::vector, kuris naudoja vientisą atminties bloką, std::deque yra sudarytas iš kelių mažesnių atminties blokų (segmentų), sujungtų per indeksavimo mechanizmą, kas šiek tiek paspartina elementų perstumimą, lyginant su std::vector.

1.14 3 strategija (pateikiamas kelių testų laikų vidurkis)

Failo dydis	std::vector	std::list	std::deque
1 000	0.0025987	0.0034226	0.0046571
10 000	0.0255509	0.0374303	0.0391001
100 000	0.276189	0.351151	0.55269
1 000 000	3.0776	4.90309	3.80778
10 000 000	33.412	43.9702	33.0299

3 strategijoje naudojamas efektyvus STL algoritmas, kuris leidžia išvengti brangių šalinimo (erase) operacijų ir sumažinti kopijavimo kiekį. Elementai yra swap'inami tame pačiame konteineryje. std::vector čia buvo greičiausias, nes turi nuoseklią atmintį, std::deque naudoja segmentuotą atmintį, o partition metu vyksta daug perrikiavimų tarp segmentų, todėl blogiau panaudojamas cache.

Bendra išvada:

3 strategija yra efektyviausia, nes ji išvengia brangių šalinimo operacijų ir sumažina kopijavimo kiekį, naudodama optimizuotus STL algoritmus. Tuo tarpu 2 strategija tampa neefektyvi dėl dažnų erase operacijų, o 1 strategija, nors ir paprasta, reikalauja papildomo kopijavimo ir atminties.

Nuotraukos, kad tyrimai įvyko:

1.15 Versija 1.1.

Buvo ištirta programos veikimo sparta su dviem skirtingomis realizacijomis: struktūra Student ir klase Student. Testavimui buvo naudojami dviejų skirtingų dydžių failai - 1 000 000 ir 10 000 000 įrašų. Tyrimas buvo daromas su trimis skirtingo optimizavimo lygio vėliavėlėmis (-O1, -O2, -O3). Tokių būdu buvo dar patikrinta, kokios įtakos jie turi programos vykdymo spartai. Tyrimo rezultatai pateikti lentelėje.

1.16 Laiko matavimų lentelė (pateikiamas kelių testų laikų vidurkis)

Versija	Failo dydis	O1 (s)	O2 (s)	O3 (s)
Struktūra	1 000 000	1.39518	1.14231	1.41373

Versija	Failo dydis	O1 (s)	O2 (s)	O3 (s)
Struktūra	10 000 000	13.0076	12.2371	13.4431
Klasė	1 000 000	1.30656	1.28801	1.30197
Klasė	10 000 000	13.7888	13.4625	13.4954

1.17 exe failų dydžių lentelė

Versija	exe failo dydis su O1 (KB)	exe failo dydis su O2 (KB)	exe failo dydis su O3 (KB)
Struktūra	459	431	429
Klasė	459	431	438

Išvada: Teoriškai -O3 optimizacija turėjo būti sparčiausia visais atvejais, tačiau praktiškai -O2 optimizavimas buvo sparčiausias, tiek su struktūra, tiek su klase abiejų failų dydžių atveju, o -O3 optimizacija net pablogina veikimą. Tarp struktūros ir klasės pagal vykdymo greitį skirtumų beveik nėra. -O3 gali turėti labai daug instrukcijų, kurios netelpa į L1 cache atmintį, tad CPU tenka laukti instrukcijų iš šiek tiek lėtesnių L2/L3, dėl to optimizacija gali būti neefektyvi. Todėl efektyviausia optimizacija dažnai gali būti su -O2 vėliavėle.

1.18 Versija 1.2.

Ši programa skirta studentų duomenų valdymui. Programos Student klasė saugo studento vardą, pavardę, namų darbų pažymius, egzamino pažymius bei apskaičiuoja galutinį įvertinimą pagal medianą ir vidurkį.

1.19 Student klasės struktūra

Kintamasis	Tipas	Aprašymas
name↩ _	std::string	Vardas
surname↩ _	std::string	Pavardė
nd↩ _	std::vector<int>	Namų darbų balai
egz↩ _	int	Egzamino balas
final↩ Avg_	double	Galutinis pagal vidurkį
final↩ Med_	double	Galutinis pagal medianą

1.20 Išvesties ir įvesties operatorių perdengimas

Klasės objektų naudojimosi patogumui buvo perdengti įvedimo ir išvedimo operatoriai.

Įvestis vykdoma per `std::istream`, todėl įvesties operatorius veikia su:

1. rankiniu įvedimu (`cin`)
2. automatiniu įvedimu (`stringstream`)
3. įvedimu iš failo (`ifstream`)

Išvestis vykdoma per std::ostream, todėl išvesties operatorius gali būti naudojamas:

1. išvedimui į ekraną (`cout`)
2. išvedimui į failą (`ofstream`)

Programa taip pat turi rankinius testus, kurie tikrina:

- konstruktorius
- kopijavimą
- move semantiką
- destruktorių
- getter'ius ir setter'ius
- įvesties ir išvesties operatorius
- skaičiavimo tikslumą

Fotografijos:

1.21 Versija 1.5.

Taip pat vėliau buvo pridėta abstrakti klasė Person. Person klasė turi čiuos kintamuosius:

Kintamasis	Tipas	Aprašymas
name↩ _	std::string	Vardas
surname↩ _	std::string	Pavardė

Tada Student klasėje lieka tik šie keturi kintamieji:

Kintamasis	Tipas	Aprašymas
nd↩ _	std::vector<int>	Namų darbų balai

Kintamasis	Tipas	Aprašymas
egz↩ _	int	Egzamino balas
final↩ Avg_	double	Galutinis pagal vidurkį
final↩ Med_	double	Galutinis pagal medianą

Kadangi ši klasė yra abstrakti, negalima sukurti šios klasės objektų. Tai pagrindžia šios klasės testas, kuris bando sukurti Person objektą, tačiau tuo atveju testavimų failas nesikompiliuoja.

Fototgrafijos, įrodančios, kad taip tikrai yra:

Programa išliko veiksni po 1.2 versijos logikos įgyvendinimo:

1.22 Versija 2.0.

Šioje projekto versijoje su Doxygen įrankiu buvo parengta dokumentacija HTML formatu ir PDF formatu per La↩ Tex, kuri aprašo projekte sukurtą klasę. Taip pat buvo realizuoti automatiniai unit testai programos klasės copy konstruktoriui, copy priskirimo operatoriui, move konstruktoriui, move priskirimo operatoriui, getteriams, setteriams, skaičiavimo logikai, polimorfizmui, išvedimo ir įvedimo operatoriams su failais. Projektui buvo sudarytas diegimo failas CMakeLists.txt, kuris atskirai generuoja paleidimo failus pačiai programai ir unit testams.

Visi Google unit testai (jų iš viso yra 12) buvo įvykdyti sėkmingai:

Klasė ir visi Rule of Five komponentai veikia teisingai, failų I/O ir skaičiavimų logika taip pat yra teisinga.

1.23 Versija 3.0.

Vektoriaus funkcijų pavyzdžių aprašymai:

1. push_back() Prideda naują elementą į vector pabaigą. Jei dabartinė talpa (capacity) yra pilna, automatiškai išskiriama nauja atmintis ir talpa padidinama (dvigubinama). Pvz.: Vector<int> v; v.push_back(1); v.push↩ _back(2);
2. erase() Pašalina elementą pagal indeksą. Visi elementai po pašalinto elemento yra perstumiami į kairę. Operacijos sudėtingumas yra O(n), nes reikia perkopijuoti likusius elementus. Pvz.: Vector<int> v; v.push↩ _back(1); v.push_back(2); v.push_back(3);

   v.erase(1);
3. resize() Pakeičia vector dydį. Jei naujas dydis didesnis, pridedami nauji elementai su default reikšme. Jei mažesnis – pertekliniai elementai pašalinami. Pvz.: Vector<int> v; v.push_back(1); v.resize(5);
4. operator[] Leidžia tiesiogiai pasiekti elementą pagal indeksą be ribų tikrinimo. Greita, bet nesaugu operacija (gali sukelti undefined behavior jei indeksas neteisingas). Pvz.: Vector<int> v; v.push_back(10); v[0] = 100;
5. at() Veikia kaip operator[], bet su papildomu vektoriaus ribų tikrinimu. Jei elemento indeksas yra už ribų, išvedama std::out_of_range žinutė. Pvz.: Vector<int> v; v.push_back(10); int x = v.at(0);

std::vector ir Vector spartos palyginimas Benchmark atliktas naudojant <chrono> biblioteka.
Matuotas push_back() operacijos laikas, užpildant tuščius vektorius.
Kiekvienas testas vykdytas atskirai, naudojant naujus (tuščius) vektorius. Kompiliuojama buvo su -O2 vėliavėle.

Elementų skaičius	std::vector (ms)	Vector (ms)	Skirtumas (ms)
10 000	0	0	0

Elementų skaičius	std::vector (ms)	Vector (ms)	Skirtumas (ms)
100 000	0	0	0
1 000 000	2	2	0
10 000 000	29	44	15
100 000 000	351	461	110

Rezultatų analizė: std::vector yra žymiai greitesnis už sukurtą Vector konteinerį. std::vector yra optimizuotas STL konteineris, kuris naudoja efektyvų atminties valdymą ir mažina reallocations skaičių. Sukurtas Vector naudoja paprastą new/delete ir rankinį kopijavimą, todėl yra lėtesnis. Didėjant elementų skaičiui, skirtumas tampa ryškesnis dėl dažnesnių atminties perskirstymų.

Realokacijų skaičiaus analizė: Buvo palygintas std::vector ir realizuoto Vector konteinerių atminties perskirstymų skaičius užpildant konteinerius 10000, 100000, 1000000, 10000000, 100000000 elementų. Atminties perskirstymas buvo laikomas įvykusiu tada, kai konteinerio capacity() reikšmė pasikeisdavo po push_back() operacijos.

Elementų skaičius	std::vector laikas	std::vector perskirstymai	Vector laikas	Vector perskirstymai
10 000	0 ms	15	0 ms	15
100 000	0 ms	18	0 ms	18
1 000 000	2 ms	21	2 ms	21
10 000 000	40 ms	25	49 ms	25
100 000 000	380 ms	28	422 ms	28

Išvados: Tiek std::vector, tiek realizuotas Vector konteineriai atliko vienodą skaičių atminties perskirstymų visais testuotais dydžiais. std::vector visais atvejais veikė šiek tiek greičiau nei realizuotas Vector.

std::vector ir sukurto konteinerio Vector spartos palyginimas:

Failo dydis	std::vector	Vector
100 000	0.25475 s	0.350412 s
1 000 000	2.64173 s	3.7302 s
10 000 000	29.3114 s	61.2496 s

Rezultatų analizė: Atlikus std::vector ir Vector realizacijos spartos palyginimą, matyti tendencija, kad visais testuotais atvejais std::vector yra greitesnis. Skirtumas tampa ypač ryškus didėjant duomenų kiekiui: esant 100 000 įrašų, skirtumas dar nedidelis, tačiau prie 10 000 000 įrašų Vector jau yra daugiau nei dvigubai lėtesnis. Tai rodo, kad standartinė bibliotekos realizacija yra stipriai optimizuota (atminties valdymas, kopijavimas, alokacijų strategijos), o rankiniu būdu sukurta Vector versija, nors funkcionaliai teisinga, spartos atžvilgiu jai neprilygsta. std::vector yra efektyvesnis pasirinkimas dideliems duomenų kiekiams.

1.24 Naudojimosi instrukcija

Įsitikinkite, kad turite įdiegtą CMake (minimali rekomenduojama versija yra 3.10). Atidarykite terminalą ir nueikite į projekto katalogą.

Programos paruošimas naudojimuisi: (šituos žingsnius reikia daryti tik pirmą kartą leidžiant programą)

1. Paruoškite programą kompiliavimui: cmake -S . -B build
2. Suompiliuokite programą (jei kažką pakeotėte savo kode, prieš paleisdami programą būtinai pakartokite šią komandą (perkompiliuokite projektą)): cmake –build build –config Release (jei kažką pakeitėte kode, prieš paleisdami programą būtinai ištrinkite pagrindiniame programos kataloge aplankalą build ir pakartokite visas komandas iš naujo)

Programos paleidimas:

Norėdami paleisti programą, terminale įveskite komandą: build\output\Release\programa.exe

Norėdami paleisti testus, terminale įveskite komandą (pasirinktinai vieną iš trijų, priklausomai nuo to, kurį testą norite paleisti): • build\Release\student_tests.exe • build\Release\vector_tests.exe • build\Release\benchmark_tests.exe

Chapter 2

Directory Hierarchy

2.1 Directories

include . . . 19
- calculations.h . . . 31
- fileHandling.h . . . 33
- inputValid.h . . . 34
- meniu.h . . . 35
- output.h . . . 37
- person.h . . . 38
- student.h . . . 39
- vector.h . . . 40

src . . . 19
- calculations.cpp . . . 44
- fileHandling.cpp . . . 45
- inputValid.cpp . . . 47
- main.cpp . . . 47
- meniu.cpp . . . 48
- output.cpp . . . 53
- person.cpp . . . 56
- student.cpp . . . 56

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Person 21
  Student 23
Vector< T > 26

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Person 21
Student 23
Vector< T > 26

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/calculations.h 31
include/fileHandling.h 33
include/inputValid.h 34
include/meniu.h 35
include/output.h 37
include/person.h 38
include/student.h 39
include/vector.h 40
src/calculations.cpp 44
src/fileHandling.cpp 45
src/inputValid.cpp 47
src/main.cpp 47
src/meniu.cpp 48
src/output.cpp 53
src/person.cpp 56
src/student.cpp 56

Chapter 6

Directory Documentation

6.1 include Directory Reference

Files

- file calculations.h
- file fileHandling.h
- file inputValid.h
- file meniu.h
- file output.h
- file person.h
- file student.h
- file vector.h

6.2 src Directory Reference

Files

- file calculations.cpp
- file fileHandling.cpp
- file inputValid.cpp
- file main.cpp
- file meniu.cpp
- file output.cpp
- file person.cpp
- file student.cpp

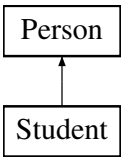
Chapter 7

Class Documentation

7.1 Person Class Reference

```
#include <person.h>
```

Inheritance diagram for Person:



Public Member Functions

- Person (const std::string &name="", const std::string &surname="")
- virtual ~Person ()=default
- Person (const Person &other)
- Person & operator= (const Person &other)
- Person (Person &&other) noexcept
- Person & operator= (Person &&other) noexcept
- virtual void print () const =0
- const std::string & getName () const
- const std::string & getSurname () const

Protected Attributes

- std::string name_
- std::string surname_

7.1.1 Constructor & Destructor Documentation

7.1.1.1 Person() [1/3]

```
Person::Person (
            const std::string & name = "",
            const std::string & surname = "")  [inline]
```

7.1.1.2 ~Person()

```
virtual Person::~Person () [virtual], [default]
```

7.1.1.3 Person() [2/3]

```
Person::Person (
            const Person & other)
```

7.1.1.4 Person() [3/3]

```
Person::Person (
            Person && other) [noexcept]
```

7.1.2 Member Function Documentation

7.1.2.1 getName()

```
const std::string & Person::getName () const [inline]
```

7.1.2.2 getSurname()

```
const std::string & Person::getSurname () const [inline]
```

7.1.2.3 operator=() [1/2]

```
Person & Person::operator= (
            const Person & other)
```

7.1.2.4 operator=() [2/2]

```
Person & Person::operator= (
            Person && other) [noexcept]
```

7.1.2.5 print()

```
virtual void Person::print () const [pure virtual]
```

Implemented in Student.

7.1.3 Member Data Documentation

7.1.3.1 name_

```
std::string Person::name_ [protected]
```

7.1.3.2 surname_

```
std::string Person::surname_ [protected]
```

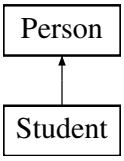
The documentation for this class was generated from the following files:

- include/person.h
- src/person.cpp

7.2 Student Class Reference

```
#include <student.h>
```

Inheritance diagram for Student:



Public Member Functions

- Student (const std::string name="Julius", const std::string surname="Petraitis", const std::vector< int > nd={1, 2}, int egz=5)
- ~Student ()
- Student (const Student &other)
- Student & operator= (const Student &other)
- Student (Student &&other) noexcept
- Student & operator= (Student &&other) noexcept
- int getEgz () const
- const std::vector< int > & getNd () const
- double getFinalAvg () const
- double getFinalMed () const
- void setNd (const std::vector< int > &nd)
- void setEgz (int egz)
- void addNd (int mark)
- void print () const override

Public Member Functions inherited from Person

- Person (const std::string &name="", const std::string &surname="")
- virtual ~Person ()=default
- Person (const Person &other)
- Person & operator= (const Person &other)
- Person (Person &&other) noexcept
- Person & operator= (Person &&other) noexcept
- const std::string & getName () const
- const std::string & getSurname () const

Friends

- std::ostream & operator<< (std::ostream &os, const Student &s)
- std::istream & operator>> (std::istream &is, Student &s)

Additional Inherited Members

Protected Attributes inherited from Person

- std::string name_
- std::string surname_

7.2.1 Constructor & Destructor Documentation

7.2.1.1 Student() [1/3]

```
Student::Student (
            const std::string name = "Julius",
            const std::string surname = "Petraitis",
            const std::vector< int > nd = {1, 2},
            int egz = 5)
```

7.2.1.2 ~Student()

```
Student::~Student ()
```

7.2.1.3 Student() [2/3]

```
Student::Student (
            const Student & other)
```

7.2.1.4 Student() [3/3]

```
Student::Student (
            Student && other)  [noexcept]
```

7.2.2 Member Function Documentation

7.2.2.1 addNd()

```
void Student::addNd (
            int mark)
```

7.2.2.2 getEgz()

```
int Student::getEgz () const
```

7.2.2.3 getFinalAvg()

```
double Student::getFinalAvg () const
```

7.2.2.4 getFinalMed()

```
double Student::getFinalMed () const
```

7.2.2.5 getNd()

```
const std::vector< int > & Student::getNd () const
```

7.2.2.6 operator=() [1/2]

```
Student & Student::operator= (
            const Student & other)
```

7.2.2.7 operator=() [2/2]

```
Student & Student::operator= (
            Student && other)  [noexcept]
```

7.2.2.8 print()

```
void Student::print () const  [override], [virtual]
```

Implements Person.

7.2.2.9 setEgz()

```
void Student::setEgz (
            int egz)
```

7.2.2.10 setNd()

```
void Student::setNd (
            const std::vector< int > & nd)
```

7.2.3 Friends And Related Symbol Documentation

7.2.3.1 operator<<

```
std::ostream & operator<< (
            std::ostream & os,
            const Student & s)  [friend]
```

7.2.3.2 operator>>

```
std::istream & operator>> (
            std::istream & is,
            Student & s)  [friend]
```

The documentation for this class was generated from the following files:

- include/student.h
- src/student.cpp

7.3 Vector< T > Class Template Reference

```
#include <vector.h>
```

Public Member Functions

- Vector ()
- ~Vector ()
- Vector (const Vector &other)
- Vector & operator= (const Vector &other)
- Vector (Vector &&other) noexcept
- Vector & operator= (Vector &&other) noexcept
- void push_back (const T &value)
- void insert (size_t index, const T &value)
- T & operator[ ] (size_t index)
- const T & operator[ ] (size_t index) const
- T & at (size_t index)
- const T & at (size_t index) const
- size_t size () const
- void pop_back ()
- T * begin ()
- T * end ()
- const T * begin () const
- const T * end () const

- void erase (size_t index)
- T * erase (T *it)
- void clear ()
- bool empty () const
- size_t capacity () const
- void reserve (size_t newCapacity)
- void resize (size_t newSize)
- T & front ()
- const T & front () const
- T & back ()
- const T & back () const
- template<typename Iter>
  void assign (Iter first, Iter last)

7.3.1 Constructor & Destructor Documentation

7.3.1.1 Vector() [1/3]

```
template<typename T>
Vector< T >::Vector ()  [inline]
```

7.3.1.2 ~Vector()

```
template<typename T>
Vector< T >::~Vector ()  [inline]
```

7.3.1.3 Vector() [2/3]

```
template<typename T>
Vector< T >::Vector (
            const Vector< T > & other)  [inline]
```

7.3.1.4 Vector() [3/3]

```
template<typename T>
Vector< T >::Vector (
            Vector< T > && other)  [inline], [noexcept]
```

7.3.2 Member Function Documentation

7.3.2.1 assign()

```
template<typename T>
template<typename Iter>
void Vector< T >::assign (
            Iter first,
            Iter last)  [inline]
```

7.3.2.2 at() [1/2]

```
template<typename T>
T & Vector< T >::at (
            size_t index)  [inline]
```

7.3.2.3 at() [2/2]

```
template<typename T>
const T & Vector< T >::at (
            size_t index) const  [inline]
```

7.3.2.4 back() [1/2]

```
template<typename T>
T & Vector< T >::back ()  [inline]
```

7.3.2.5 back() [2/2]

```
template<typename T>
const T & Vector< T >::back () const  [inline]
```

7.3.2.6 begin() [1/2]

```
template<typename T>
T * Vector< T >::begin ()  [inline]
```

7.3.2.7 begin() [2/2]

```
template<typename T>
const T * Vector< T >::begin () const  [inline]
```

7.3.2.8 capacity()

```
template<typename T>
size_t Vector< T >::capacity () const  [inline]
```

7.3.2.9 clear()

```
template<typename T>
void Vector< T >::clear ()  [inline]
```

7.3.2.10 empty()

```
template<typename T>
bool Vector< T >::empty () const  [inline]
```

7.3.2.11 end() [1/2]

```
template<typename T>
T * Vector< T >::end ()  [inline]
```

7.3.2.12 end() [2/2]

```
template<typename T>
const T * Vector< T >::end () const  [inline]
```

7.3.2.13 erase() [1/2]

```
template<typename T>
void Vector< T >::erase (
            size_t index)  [inline]
```

7.3.2.14 erase() [2/2]

```
template<typename T>
T * Vector< T >::erase (
            T * it)  [inline]
```

7.3.2.15 front() [1/2]

```
template<typename T>
T & Vector< T >::front ()  [inline]
```

7.3.2.16 front() [2/2]

```
template<typename T>
const T & Vector< T >::front () const  [inline]
```

7.3.2.17 insert()

```
template<typename T>
void Vector< T >::insert (
            size_t index,
            const T & value)  [inline]
```

7.3.2.18 operator=() [1/2]

```
template<typename T>
Vector & Vector< T >::operator= (
            const Vector< T > & other)  [inline]
```

7.3.2.19 operator=() [2/2]

```
template<typename T>
Vector & Vector< T >::operator= (
            Vector< T > && other)  [inline], [noexcept]
```

7.3.2.20 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
            size_t index)  [inline]
```

7.3.2.21 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
            size_t index) const  [inline]
```

7.3.2.22 pop_back()

```
template<typename T>
void Vector< T >::pop_back ()  [inline]
```

7.3.2.23 push_back()

```
template<typename T>
void Vector< T >::push_back (
            const T & value)  [inline]
```

7.3.2.24 reserve()

```
template<typename T>
void Vector< T >::reserve (
            size_t newCapacity)  [inline]
```

7.3.2.25 resize()

```
template<typename T>
void Vector< T >::resize (
            size_t newSize)  [inline]
```

7.3.2.26 size()

```
template<typename T>
size_t Vector< T >::size () const  [inline]
```

The documentation for this class was generated from the following file:

- include/vector.h

Chapter 8

File Documentation

8.1 include/calculations.h File Reference

```
#include <vector>
#include "student.h"
```

Functions

- double med (const std::vector< int > &nd, int egz)
- double avg (const std::vector< int > &nd, int egz)
- std::string GenerateName (const std::string &vardas, const std::vector< std::string > &saknys)
- bool compareByName (const Student &a, const Student &b)
- bool compareBySurname (const Student &a, const Student &b)
- bool compareByMed (const Student &a, const Student &b)
- bool compareByAvg (const Student &a, const Student &b)

8.1.1 Function Documentation

8.1.1.1 avg()

```
double avg (
            const std::vector< int > & nd,
            int egz)
```

8.1.1.2 compareByAvg()

```
bool compareByAvg (
            const Student & a,
            const Student & b)
```

8.1.1.3 compareByMed()

```
bool compareByMed (
            const Student & a,
            const Student & b)
```

8.1.1.4 compareByName()

```
bool compareByName (
            const Student & a,
            const Student & b)
```

8.1.1.5 compareBySurname()

```
bool compareBySurname (
            const Student & a,
            const Student & b)
```

8.1.1.6 GenerateName()

```
std::string GenerateName (
            const std::string & vardas,
            const std::vector< std::string > & saknys)
```

8.1.1.7 med()

```
double med (
            const std::vector< int > & nd,
            int egz)
```

8.2 calculations.h

Go to the documentation of this file.

```
#ifndef CALCULATIONS_H
#define CALCULATIONS_H

#include <vector>
#include "student.h"

double med(const std::vector<int>& nd, int egz);
double avg(const std::vector<int>& nd, int egz);
std::string GenerateName(const std::string& vardas, const std::vector<std::string>& saknys);

bool compareByName(const Student &a, const Student &b);
bool compareBySurname(const Student &a, const Student &b);
bool compareByMed(const Student &a, const Student &b);
bool compareByAvg(const Student &a, const Student &b);

#endif
```

8.3 include/fileHandling.h File Reference

```
#include <vector>
#include <list>
#include <deque>
#include <string>
#include "student.h"
```

Functions

- template<typename Container>
  bool createFile (const Container &Students, int n)
- template<typename Container>
  bool readFile (const std::string &filename, Container &Students)

8.3.1 Function Documentation

8.3.1.1 createFile()

```
template<typename Container>
bool createFile (
            const Container & Students,
            int n)
```

8.3.1.2 readFile()

```
template<typename Container>
bool readFile (
            const std::string & filename,
            Container & Students)
```

8.4 fileHandling.h

Go to the documentation of this file.

```
00001 #ifndef FILEHANDLING_H
00002 #define FILEHANDLING_H
00003
00004 #include <vector>
00005 #include <list>
00006 #include <deque>
00007 #include <string>
00008 #include "student.h"
00009
00010 template <typename Container>
00011 bool createFile(const Container& Students, int n);
00012 template <typename Container>
00013 bool readFile(const std::string& filename, Container& Students);
00014
00015 #endif
```

8.5 include/inputValid.h File Reference

```
#include <string>
```

Functions

- char inputLetter (const std::string &message)
- std::string inputWord (const std::string &message)
- int inputInt (const std::string &message, int min=INT_MIN, int max=INT_MAX)
- double inputR (const std::string &message, double min=INT_MIN, double max=INT_MAX)

8.5.1 Function Documentation

8.5.1.1 inputInt()

```
int inputInt (
            const std::string & message,
            int min = INT_MIN,
            int max = INT_MAX)
```

8.5.1.2 inputLetter()

```
char inputLetter (
            const std::string & message)
```

8.5.1.3 inputR()

```
double inputR (
            const std::string & message,
            double min = INT_MIN,
            double max = INT_MAX)
```

8.5.1.4 inputWord()

```
std::string inputWord (
            const std::string & message)
```

8.6 inputValid.h

Go to the documentation of this file.

```
#ifndef INPUTVALID_H
#define INPUTVALID_H

#include <string>

char inputLetter(const std::string& message);
std::string inputWord(const std::string& message);
int inputInt(const std::string& message, int min = INT_MIN, int max = INT_MAX);
double inputR(const std::string& message, double min = INT_MIN, double max = INT_MAX);

#endif
```

8.7 include/meniu.h File Reference

```
#include <vector>
#include <list>
#include <deque>
#include <string>
#include "student.h"
```

Functions

- int showMeniu ()
- template<typename Container>
  void manualInput (Container &Students)
- template<typename Container>
  void generateGrades (Container &Students)
- template<typename Container>
  void generateNamesGrades (Container &Students)
- template<typename Container>
  void scanFile (Container &Students)
- bool generateFile ()
- template<typename Container>
  void sortingStudents (Container &Students, bool benchmarkMode=false, const std::string &filename="")
- template<typename Container>
  void handleOutput (Container &)
- template<typename Container>
  bool readFile (const std::string &, Container &)

8.7.1 Function Documentation

8.7.1.1 generateFile()

```
bool generateFile ()
```

8.7.1.2 generateGrades()

```
template<typename Container>
void generateGrades (
            Container & Students)
```

8.7.1.3 generateNamesGrades()

```
template<typename Container>
void generateNamesGrades (
            Container & Students)
```

8.7.1.4 handleOutput()

```
template<typename Container>
void handleOutput (
            Container & Students)
```

8.7.1.5 manualInput()

```
template<typename Container>
void manualInput (
            Container & Students)
```

8.7.1.6 readFile()

```
template<typename Container>
bool readFile (
            const std::string & filename,
            Container & Students)
```

8.7.1.7 scanFile()

```
template<typename Container>
void scanFile (
            Container & Students)
```

8.7.1.8 showMeniu()

```
int showMeniu ()
```

8.7.1.9 sortingStudents()

```
template<typename Container>
void sortingStudents (
            Container & Students,
            bool benchmarkMode = false,
            const std::string & filename = "")
```

8.8 meniu.h

Go to the documentation of this file.

```
#ifndef MENIU_H
#define MENIU_H

#include <vector>
#include <list>
#include <deque>
#include <string>
#include "student.h"

int showMeniu();
template <typename Container>
void manualInput(Container& Students);
template <typename Container>
void generateGrades(Container& Students);
template <typename Container>
void generateNamesGrades(Container& Students);
template <typename Container>
void scanFile(Container& Students);
bool generateFile();

template <typename Container>
void sortingStudents(Container& Students, bool benchmarkMode = false, const std::string& filename = "");

template <typename Container>
void handleOutput(Container&);

template <typename Container>
bool readFile(const std::string&, Container&);

#endif
```

8.9 include/output.h File Reference

```
#include <vector>
#include <list>
#include <deque>
#include <string>
#include "student.h"
```

Functions

- template<typename Container>
  void printResults (const Container &Students, bool toFile=false, const std::string &filename="rezultatai.txt")
- template<typename Container>
  void outputToFileSorting (Container &Students)
- template<typename Container>
  void handleOutput (Container &Students)

8.9.1 Function Documentation

8.9.1.1 handleOutput()

```
template<typename Container>
void handleOutput (
            Container & Students)
```

8.9.1.2 outputToFileSorting()

```
template<typename Container>
void outputToFileSorting (
            Container & Students)
```

8.9.1.3 printResults()

```
template<typename Container>
void printResults (
            const Container & Students,
            bool toFile = false,
            const std::string & filename = "rezultatai.txt")
```

8.10 output.h

Go to the documentation of this file.

```
00001 #ifndef OUTPUT_H
00002 #define OUTPUT_H
00003
00004 #include <vector>
00005 #include <list>
00006 #include <deque>
00007 #include <string>
00008 #include "student.h"
00009
00010 template <typename Container>
00011 void printResults(const Container& Students, bool toFile = false, const std::string& filename =
      "rezultatai.txt");
00012 template <typename Container>
00013 void outputToFileSorting(Container& Students);
00014 template <typename Container>
00015 void handleOutput(Container& Students);
00016
00017 #endif
```

8.11 include/person.h File Reference

```
#include <string>
```

Classes

- class Person

8.12 person.h

Go to the documentation of this file.

```
#ifndef PERSON_H
#define PERSON_H

#include <string>

class Person {
protected:
    std::string name_;
    std::string surname_;

public:
    Person(const std::string& name = "", const std::string& surname = "")
        : name_(name), surname_(surname) {}

    virtual ~Person() = default;

    Person(const Person& other);
    Person& operator=(const Person& other);

    Person(Person&& other) noexcept;
    Person& operator=(Person&& other) noexcept;

    virtual void print() const = 0;

    const std::string& getName() const { return name_; }
    const std::string& getSurname() const { return surname_; }
};

#endif
```

8.13 include/student.h File Reference

```
#include "person.h"
#include <vector>
#include <iostream>
```

Classes

- class Student

8.14 student.h

Go to the documentation of this file.

```
#ifndef STUDENT_H
#define STUDENT_H

#include "person.h"
#include <vector>
#include <iostream>

class Student : public Person {
private:
    std::vector<int> nd_;
    int egz_;

    double finalAvg_;
    double finalMed_;

    void calculate();

public:
    Student(const std::string name = "Julius", const std::string surname = "Petraitis", const
std::vector<int> nd = {1, 2}, int egz = 5);
```

```

    ~Student();

    Student(const Student& other);
    Student& operator=(const Student& other);

    Student(Student&& other) noexcept;
    Student& operator=(Student&& other) noexcept;

    int getEgz() const;
    const std::vector<int>& getNd() const;

    double getFinalAvg() const;
    double getFinalMed() const;

    void setNd(const std::vector<int>& nd);
    void setEgz(int egz);
    void addNd(int mark);

    void print() const override;

    friend std::ostream& operator«(std::ostream& os, const Student& s);
    friend std::istream& operator»(std::istream& is, Student& s);
};

#endif
```

8.15 include/vector.h File Reference

Classes

- class Vector< T >

8.16 vector.h

Go to the documentation of this file.

```
#ifndef VECTOR_H
#define VECTOR_H

template<typename T>
class Vector
{
private:
    T* data_;
    size_t size_;
    size_t capacity_;

    void reallocate(size_t newCapacity) {
        T* newData = new T[newCapacity];

        for(size_t i = 0; i < size_; i++)
        {
            newData[i] = data_[i];
        }

        delete[] data_;

        data_ = newData;
        capacity_ = newCapacity;
    }

public:
    Vector()
    {
        data_ = nullptr;
        size_ = 0;
        capacity_ = 0;
    }

    ~Vector()
    {
        delete[] data_;
        data_ = nullptr;
        size_ = 0;
```

```
        capacity_ = 0;
    }

    Vector(const Vector& other)
    {
        size_ = other.size_;
        capacity_ = other.capacity_;

        data_ = new T[capacity_];

        for(size_t i = 0; i < size_; i++)
        {
            data_[i] = other.data_[i];
        }
    }

    Vector& operator=(const Vector& other)
    {
        if(this == &other)
        {
            return *this;
        }

        delete[] data_;

        size_ = other.size_;
        capacity_ = other.capacity_;

        data_ = new T[capacity_];

        for(size_t i = 0; i < size_; i++)
        {
            data_[i] = other.data_[i];
        }

        return *this;
    }

    Vector(Vector&& other) noexcept
    {
        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;

        other.data_ = nullptr;
        other.size_ = 0;
        other.capacity_ = 0;
    }

    Vector& operator=(Vector&& other) noexcept
    {
        if(this == &other)
        {
            return *this;
        }

        delete[] data_;

        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;

        other.data_ = nullptr;
        other.size_ = 0;
        other.capacity_ = 0;

        return *this;
    }

    void push_back(const T& value) {
        if(size_ >= capacity_) {
            size_t newCapacity;
            if (capacity_ == 0) { newCapacity = 1; }
            else { newCapacity = capacity_ * 2; }
            reallocate(newCapacity);
        }

        data_[size_] = value;
        size_++;
    }

    void insert(size_t index, const T& value) {
        if(index > size_)
        {
            throw std::out_of_range("Index out of range");
        }

```

```
        if(size_ >= capacity_)
        {
            size_t newCapacity;
            if (capacity_ == 0) { newCapacity = 1; }
            else { newCapacity = capacity_ * 2; }
            reallocate(newCapacity);
        }

        for(size_t i = size_; i > index; i--)
        {
            data_[i] = data_[i - 1];
        }

        data_[index] = value;
        size_++;
    }


    T& operator[](size_t index) { return data_[index]; }

    const T& operator[](size_t index) const { return data_[index]; }

    T& at(size_t index) {
        if (index >= size_) { throw std::out_of_range("Index out of range"); }
        return data_[index];
    }

    const T& at(size_t index) const {
        if (index >= size_) { throw std::out_of_range("Index out of range"); }
        return data_[index];
    }

    size_t size() const { return size_; }

    void pop_back()
    {
        if(size_ > 0)
        {
            size_--;
        }
    }

    T* begin() { return data_; }

    T* end() { return data_ + size_; }

    const T* begin() const { return data_; }

    const T* end() const { return data_ + size_; }

    void erase(size_t index) {
        if (index >= size_)
        {
            throw std::out_of_range("Index out of range");
        }

        for (size_t i = index; i < size_ - 1; i++)
        {
            data_[i] = data_[i + 1];
        }

        size_--;
    }

    T* erase(T* it)
    {
        size_t index = it - data_;

        if (index >= size_)
            throw std::out_of_range("Index out of range");

        for (size_t i = index; i < size_ - 1; i++)
            data_[i] = data_[i + 1];

        size_--;

        return data_ + index;
    }

    void clear() { size_ = 0; }

    bool empty() const { return size_ == 0; }

    size_t capacity() const { return capacity_; }

    void reserve(size_t newCapacity) {
        if (newCapacity > capacity_)
```

```
        {
            reallocate(newCapacity);
        }
    }

    void resize(size_t newSize) {
        if (newSize > capacity_)
        {
            size_t newCapacity;

            if (capacity_ == 0)
            {
                newCapacity = 1;
            }
            else
            {
                newCapacity = capacity_;
            }

            while (newCapacity < newSize)
            {
                newCapacity *= 2;
            }

            reallocate(newCapacity);
        }

        for (size_t i = size_; i < newSize; i++)
        {
            data_[i] = T();
        }

        size_ = newSize;
    }

    T& front() {
        if(empty())
        {
            throw std::out_of_range("Vector is empty");
        }
        return data_[0];
    }

    const T& front() const {
        if(empty())
        {
            throw std::out_of_range("Vector is empty");
        }
        return data_[0];
    }

    T& back() {
        if(empty())
        {
            throw std::out_of_range("Vector is empty");
        }
        return data_[size_ - 1];
    }

    const T& back() const {
        if(empty())
        {
            throw std::out_of_range("Vector is empty");
        }
        return data_[size_ - 1];
    }

    template <typename Iter>
    void assign(Iter first, Iter last)
    {
        clear();

        for (; first != last; ++first)
            push_back(*first);
    }

};

#endif
```

8.17 README.md File Reference

8.18 src/calculations.cpp File Reference

```
#include "calculations.h"
#include <algorithm>
#include <random>
```

Functions

- std::string GenerateName (const std::string &vardas, const std::vector< std::string > &saknys)
- bool compareByName (const Student &a, const Student &b)
- bool compareBySurname (const Student &a, const Student &b)
- bool compareByMed (const Student &a, const Student &b)
- bool compareByAvg (const Student &a, const Student &b)

8.18.1 Function Documentation

8.18.1.1 compareByAvg()

```
bool compareByAvg (
            const Student & a,
            const Student & b)
```

8.18.1.2 compareByMed()

```
bool compareByMed (
            const Student & a,
            const Student & b)
```

8.18.1.3 compareByName()

```
bool compareByName (
            const Student & a,
            const Student & b)
```

8.18.1.4 compareBySurname()

```
bool compareBySurname (
            const Student & a,
            const Student & b)
```

8.18.1.5 GenerateName()

```
std::string GenerateName (
            const std::string & vardas,
            const std::vector< std::string > & saknys)
```

8.19 src/fileHandling.cpp File Reference

```
#include "fileHandling.h"
#include "calculations.h"
#include "inputValid.h"
#include "vector.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <iomanip>
#include <vector>
#include <list>
#include <deque>
```

Functions

- template<typename Container>
  bool createFile (const Container &Students, int n)
- template<typename Container>
  bool readFile (const std::string &filename, Container &Students)
- template bool createFile< std::vector< Student > > (const std::vector< Student > &, int)
- template bool createFile< std::list< Student > > (const std::list< Student > &, int)
- template bool createFile< std::deque< Student > > (const std::deque< Student > &, int)
- template bool readFile< std::vector< Student > > (const std::string &, std::vector< Student > &)
- template bool readFile< std::list< Student > > (const std::string &, std::list< Student > &)
- template bool readFile< std::deque< Student > > (const std::string &, std::deque< Student > &)
- template bool readFile< Vector< Student > > (const std::string &, Vector< Student > &)

8.19.1 Function Documentation

8.19.1.1 createFile()

```
template<typename Container>
bool createFile (
            const Container & Students,
            int n)
```

8.19.1.2 createFile< std::deque< Student > >()

```
template bool createFile< std::deque< Student > > (
            const std::deque< Student > & ,
            int )
```

8.19.1.3 createFile< std::list< Student > >()

```
template bool createFile< std::list< Student > > (
            const std::list< Student > & ,
            int )
```

8.19.1.4 createFile< std::vector< Student > >()

```
template bool createFile< std::vector< Student > > (
            const std::vector< Student > & ,
            int )
```

8.19.1.5 readFile()

```
template<typename Container>
bool readFile (
            const std::string & filename,
            Container & Students)
```

8.19.1.6 readFile< std::deque< Student > >()

```
template bool readFile< std::deque< Student > > (
            const std::string & ,
            std::deque< Student > & )
```

8.19.1.7 readFile< std::list< Student > >()

```
template bool readFile< std::list< Student > > (
            const std::string & ,
            std::list< Student > & )
```

8.19.1.8 readFile< std::vector< Student > >()

```
template bool readFile< std::vector< Student > > (
            const std::string & ,
            std::vector< Student > & )
```

8.19.1.9 readFile< Vector< Student > >()

```
template bool readFile< Vector< Student > > (
            const std::string & ,
            Vector< Student > & )
```

8.20 src/inputValid.cpp File Reference

```
#include "inputValid.h"
#include <iostream>
```

Functions

- char inputLetter (const std::string &message)
- std::string inputWord (const std::string &message)
- int inputInt (const std::string &message, int min, int max)
- double inputR (const std::string &message, double min, double max)

8.20.1 Function Documentation

8.20.1.1 inputInt()

```
int inputInt (
            const std::string & message,
            int min,
            int max)
```

8.20.1.2 inputLetter()

```
char inputLetter (
            const std::string & message)
```

8.20.1.3 inputR()

```
double inputR (
            const std::string & message,
            double min,
            double max)
```

8.20.1.4 inputWord()

```
std::string inputWord (
            const std::string & message)
```

8.21 src/main.cpp File Reference

```
#include "meniu.h"
#include "output.h"
#include "inputValid.h"
#include "vector.h"
#include <vector>
#include <ctime>
#include <list>
#include <deque>
```

Functions

- int main ()

8.21.1 Function Documentation

8.21.1.1 main()

```
int main ()
```

8.22 src/meniu.cpp File Reference

```
#include "meniu.h"
#include "inputValid.h"
#include "calculations.h"
#include "output.h"
#include "fileHandling.h"
#include "vector.h"
#include <iostream>
#include <chrono>
#include <iomanip>
#include <random>
#include <fstream>
#include <algorithm>
#include <vector>
#include <list>
#include <deque>
#include <string>
```

Functions

- int showMeniu ()
- template<typename Container>
  void manualInput (Container &Students)
- template<typename Container>
  void generateGrades (Container &Students)
- template<typename Container>
  void generateNamesGrades (Container &Students)
- template<typename Container>
  void scanFile (Container &Students)
- bool generateFile ()
- template<typename Container>
  void sortingStudents (Container &students, bool benchmarkMode, const std::string &filename)
- template void manualInput< std::vector< Student > > (std::vector< Student > &)
- template void manualInput< std::list< Student > > (std::list< Student > &)
- template void manualInput< std::deque< Student > > (std::deque< Student > &)
- template void generateGrades< std::vector< Student > > (std::vector< Student > &)
- template void generateGrades< std::list< Student > > (std::list< Student > &)
- template void generateGrades< std::deque< Student > > (std::deque< Student > &)

- template void generateNamesGrades< std::vector< Student > > (std::vector< Student > &)
- template void generateNamesGrades< std::list< Student > > (std::list< Student > &)
- template void generateNamesGrades< std::deque< Student > > (std::deque< Student > &)
- template void scanFile< std::vector< Student > > (std::vector< Student > &)
- template void scanFile< std::list< Student > > (std::list< Student > &)
- template void scanFile< std::deque< Student > > (std::deque< Student > &)
- template void manualInput< Vector< Student > > (Vector< Student > &)
- template void generateGrades< Vector< Student > > (Vector< Student > &)
- template void generateNamesGrades< Vector< Student > > (Vector< Student > &)
- template void scanFile< Vector< Student > > (Vector< Student > &)
- template void handleOutput< Vector< Student > > (Vector< Student > &)
- template bool readFile< Vector< Student > > (const std::string &, Vector< Student > &)
- template void sortingStudents< std::vector< Student > > (std::vector< Student > &, bool, const std::string &)
- template void sortingStudents< std::list< Student > > (std::list< Student > &, bool, const std::string &)
- template void sortingStudents< std::deque< Student > > (std::deque< Student > &, bool, const std::string &)
- template void sortingStudents< Vector< Student > > (Vector< Student > &, bool, const std::string &)

8.22.1 Function Documentation

8.22.1.1 generateFile()

```
bool generateFile ()
```

8.22.1.2 generateGrades()

```
template<typename Container>
void generateGrades (
            Container & Students)
```

8.22.1.3 generateGrades< std::deque< Student > >()

```
template void generateGrades< std::deque< Student > > (
            std::deque< Student > & )
```

8.22.1.4 generateGrades< std::list< Student > >()

```
template void generateGrades< std::list< Student > > (
            std::list< Student > & )
```

8.22.1.5 generateGrades< std::vector< Student > >()

```
template void generateGrades< std::vector< Student > > (
            std::vector< Student > & )
```

8.22.1.6 generateGrades< Vector< Student > >()

```
template void generateGrades< Vector< Student > > (
            Vector< Student > & )
```

8.22.1.7 generateNamesGrades()

```
template<typename Container>
void generateNamesGrades (
            Container & Students)
```

8.22.1.8 generateNamesGrades< std::deque< Student > >()

```
template void generateNamesGrades< std::deque< Student > > (
            std::deque< Student > & )
```

8.22.1.9 generateNamesGrades< std::list< Student > >()

```
template void generateNamesGrades< std::list< Student > > (
            std::list< Student > & )
```

8.22.1.10 generateNamesGrades< std::vector< Student > >()

```
template void generateNamesGrades< std::vector< Student > > (
            std::vector< Student > & )
```

8.22.1.11 generateNamesGrades< Vector< Student > >()

```
template void generateNamesGrades< Vector< Student > > (
            Vector< Student > & )
```

8.22.1.12 handleOutput< Vector< Student > >()

```
template void handleOutput< Vector< Student > > (
            Vector< Student > & )
```

8.22.1.13 manualInput()

```
template<typename Container>
void manualInput (
            Container & Students)
```

8.22.1.14 manualInput< std::deque< Student > >()

```
template void manualInput< std::deque< Student > > (
            std::deque< Student > & )
```

8.22.1.15 manualInput< std::list< Student > >()

```
template void manualInput< std::list< Student > > (
            std::list< Student > & )
```

8.22.1.16 manualInput< std::vector< Student > >()

```
template void manualInput< std::vector< Student > > (
            std::vector< Student > & )
```

8.22.1.17 manualInput< Vector< Student > >()

```
template void manualInput< Vector< Student > > (
            Vector< Student > & )
```

8.22.1.18 readFile< Vector< Student > >()

```
template bool readFile< Vector< Student > > (
            const std::string & ,
            Vector< Student > & )
```

8.22.1.19 scanFile()

```
template<typename Container>
void scanFile (
            Container & Students)
```

8.22.1.20 scanFile< std::deque< Student > >()

```
template void scanFile< std::deque< Student > > (
            std::deque< Student > & )
```

8.22.1.21 scanFile< std::list< Student > >()

```
template void scanFile< std::list< Student > > (
            std::list< Student > & )
```

8.22.1.22 scanFile< std::vector< Student > >()

```
template void scanFile< std::vector< Student > > (
            std::vector< Student > & )
```

8.22.1.23 scanFile< Vector< Student > >()

```
template void scanFile< Vector< Student > > (
            Vector< Student > & )
```

8.22.1.24 showMeniu()

```
int showMeniu ()
```

8.22.1.25 sortingStudents()

```
template<typename Container>
void sortingStudents (
            Container & students,
            bool benchmarkMode,
            const std::string & filename)
```

8.22.1.26 sortingStudents< std::deque< Student > >()

```
template void sortingStudents< std::deque< Student > > (
            std::deque< Student > & ,
            bool ,
            const std::string & )
```

8.22.1.27 sortingStudents< std::list< Student > >()

```
template void sortingStudents< std::list< Student > > (
            std::list< Student > & ,
            bool ,
            const std::string & )
```

8.22.1.28 sortingStudents< std::vector< Student > >()

```
template void sortingStudents< std::vector< Student > > (
            std::vector< Student > & ,
            bool ,
            const std::string & )
```

8.22.1.29 sortingStudents< Vector< Student > >()

```
template void sortingStudents< Vector< Student > > (
            Vector< Student > & ,
            bool ,
            const std::string & )
```

8.23 src/output.cpp File Reference

```
#include "output.h"
#include "inputValid.h"
#include "calculations.h"
#include "vector.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <fstream>
#include <vector>
#include <list>
#include <deque>
```

Functions

- template<typename Container, typename Compare>
  void sortContainer (Container &c, Compare comp)
- template<typename Compare>
  void sortContainer (std::list< Student > &c, Compare comp)
- template<typename Container>
  void printResults (const Container &Students, bool toFile, const std::string &filename)
- template<typename Container>
  void outputToFileSorting (Container &Students)
- template<typename Container>
  void handleOutput (Container &Students)
- template void printResults< std::vector< Student > > (const std::vector< Student > &, bool, const std::↩ string &)
- template void printResults< std::list< Student > > (const std::list< Student > &, bool, const std::string &)
- template void printResults< std::deque< Student > > (const std::deque< Student > &, bool, const std::↩ string &)
- template void outputToFileSorting< std::vector< Student > > (std::vector< Student > &)
- template void outputToFileSorting< std::list< Student > > (std::list< Student > &)
- template void outputToFileSorting< std::deque< Student > > (std::deque< Student > &)
- template void handleOutput< std::vector< Student > > (std::vector< Student > &)
- template void handleOutput< std::list< Student > > (std::list< Student > &)
- template void handleOutput< std::deque< Student > > (std::deque< Student > &)
- template void printResults< Vector< Student > > (const Vector< Student > &, bool, const std::string &)
- template void outputToFileSorting< Vector< Student > > (Vector< Student > &)
- template void handleOutput< Vector< Student > > (Vector< Student > &)

8.23.1 Function Documentation

8.23.1.1 handleOutput()

```
template<typename Container>
void handleOutput (
            Container & Students)
```

8.23.1.2 handleOutput< std::deque< Student > >()

```
template void handleOutput< std::deque< Student > > (
            std::deque< Student > & )
```

8.23.1.3 handleOutput< std::list< Student > >()

```
template void handleOutput< std::list< Student > > (
            std::list< Student > & )
```

8.23.1.4 handleOutput< std::vector< Student > >()

```
template void handleOutput< std::vector< Student > > (
            std::vector< Student > & )
```

8.23.1.5 handleOutput< Vector< Student > >()

```
template void handleOutput< Vector< Student > > (
            Vector< Student > & )
```

8.23.1.6 outputToFileSorting()

```
template<typename Container>
void outputToFileSorting (
            Container & Students)
```

8.23.1.7 outputToFileSorting< std::deque< Student > >()

```
template void outputToFileSorting< std::deque< Student > > (
            std::deque< Student > & )
```

8.23.1.8 outputToFileSorting< std::list< Student > >()

```
template void outputToFileSorting< std::list< Student > > (
            std::list< Student > & )
```

8.23.1.9 outputToFileSorting< std::vector< Student > >()

```
template void outputToFileSorting< std::vector< Student > > (
            std::vector< Student > & )
```

8.23.1.10 outputToFileSorting< Vector< Student > >()

```
template void outputToFileSorting< Vector< Student > > (
            Vector< Student > & )
```

8.23.1.11 printResults()

```
template<typename Container>
void printResults (
            const Container & Students,
            bool toFile,
            const std::string & filename)
```

8.23.1.12 printResults< std::deque< Student > >()

```
template void printResults< std::deque< Student > > (
            const std::deque< Student > & ,
            bool ,
            const std::string & )
```

8.23.1.13 printResults< std::list< Student > >()

```
template void printResults< std::list< Student > > (
            const std::list< Student > & ,
            bool ,
            const std::string & )
```

8.23.1.14 printResults< std::vector< Student > >()

```
template void printResults< std::vector< Student > > (
            const std::vector< Student > & ,
            bool ,
            const std::string & )
```

8.23.1.15 printResults< Vector< Student > >()

```
template void printResults< Vector< Student > > (
            const Vector< Student > & ,
            bool ,
            const std::string & )
```

8.23.1.16 sortContainer() [1/2]

```
template<typename Container, typename Compare>
void sortContainer (
            Container & c,
            Compare comp)
```

8.23.1.17 sortContainer() [2/2]

```
template<typename Compare>
void sortContainer (
            std::list< Student > & c,
            Compare comp)
```

8.24 src/person.cpp File Reference

```
#include "../include/person.h"
```

8.25 src/student.cpp File Reference

```
#include "student.h"
#include <algorithm>
#include <iostream>
```

Functions

- std::ostream & operator<< (std::ostream &os, const Student &s)
- std::istream & operator>> (std::istream &is, Student &s)

8.25.1 Function Documentation

8.25.1.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & os,
            const Student & s)
```

8.25.1.2 operator>>()

```
std::istream & operator>> (
            std::istream & is,
            Student & s)
```

Index

~Person
  Person, 21
~Student
  Student, 24
~Vector
  Vector< T >, 27

addNd
  Student, 25
Aprašymas, 1
assign
  Vector< T >, 27
at
  Vector< T >, 27, 28
avg
  calculations.h, 31

back
  Vector< T >, 28
begin
  Vector< T >, 28

calculations.cpp
  compareByAvg, 44
  compareByMed, 44
  compareByName, 44
  compareBySurname, 44
  GenerateName, 44
calculations.h
  avg, 31
  compareByAvg, 31
  compareByMed, 31
  compareByName, 32
  compareBySurname, 32
  GenerateName, 32
  med, 32
capacity
  Vector< T >, 28
clear
  Vector< T >, 28
compareByAvg
  calculations.cpp, 44
  calculations.h, 31
compareByMed
  calculations.cpp, 44
  calculations.h, 31
compareByName
  calculations.cpp, 44
  calculations.h, 32
compareBySurname
  calculations.cpp, 44
  calculations.h, 32
createFile
  fileHandling.cpp, 45
  fileHandling.h, 33
createFile< std::deque< Student > >
  fileHandling.cpp, 45
createFile< std::list< Student > >
  fileHandling.cpp, 45
createFile< std::vector< Student > >
  fileHandling.cpp, 46

empty
  Vector< T >, 28
end
  Vector< T >, 29
erase
  Vector< T >, 29

fileHandling.cpp
  createFile, 45
  createFile< std::deque< Student > >, 45
  createFile< std::list< Student > >, 45
  createFile< std::vector< Student > >, 46
  readFile, 46
  readFile< std::deque< Student > >, 46
  readFile< std::list< Student > >, 46
  readFile< std::vector< Student > >, 46
  readFile< Vector< Student > >, 46
fileHandling.h
  createFile, 33
  readFile, 33
front
  Vector< T >, 29

generateFile
  meniu.cpp, 49
  meniu.h, 35
generateGrades
  meniu.cpp, 49
  meniu.h, 35
generateGrades< std::deque< Student > >
  meniu.cpp, 49
generateGrades< std::list< Student > >
  meniu.cpp, 49
generateGrades< std::vector< Student > >
  meniu.cpp, 49
generateGrades< Vector< Student > >
  meniu.cpp, 49
GenerateName

calculations.cpp, 44
calculations.h, 32
generateNamesGrades
meniu.cpp, 50
meniu.h, 35
generateNamesGrades< std::deque< Student > >
meniu.cpp, 50
generateNamesGrades< std::list< Student > >
meniu.cpp, 50
generateNamesGrades< std::vector< Student > >
meniu.cpp, 50
generateNamesGrades< Vector< Student > >
meniu.cpp, 50
getEgz
Student, 25
getFinalAvg
Student, 25
getFinalMed
Student, 25
getName
Person, 22
getNd
Student, 25
getSurname
Person, 22

handleOutput
meniu.h, 35
output.cpp, 54
output.h, 37
handleOutput< std::deque< Student > >
output.cpp, 54
handleOutput< std::list< Student > >
output.cpp, 54
handleOutput< std::vector< Student > >
output.cpp, 54
handleOutput< Vector< Student > >
meniu.cpp, 50
output.cpp, 54

include Directory Reference, 19
include/calculations.h, 31, 32
include/fileHandling.h, 33
include/inputValid.h, 34
include/meniu.h, 35, 37
include/output.h, 37, 38
include/person.h, 38, 39
include/student.h, 39
include/vector.h, 40
inputInt
inputValid.cpp, 47
inputValid.h, 34
inputLetter
inputValid.cpp, 47
inputValid.h, 34
inputR
inputValid.cpp, 47
inputValid.h, 34
inputValid.cpp
inputInt, 47
inputLetter, 47
inputR, 47
inputWord, 47
inputValid.h
inputInt, 34
inputLetter, 34
inputR, 34
inputWord, 34
inputWord
inputValid.cpp, 47
inputValid.h, 34
insert
Vector< T >, 29

main
main.cpp, 48
main.cpp
main, 48
manualInput
meniu.cpp, 50
meniu.h, 36
manualInput< std::deque< Student > >
meniu.cpp, 50
manualInput< std::list< Student > >
meniu.cpp, 51
manualInput< std::vector< Student > >
meniu.cpp, 51
manualInput< Vector< Student > >
meniu.cpp, 51
med
calculations.h, 32
meniu.cpp
generateFile, 49
generateGrades, 49
generateGrades< std::deque< Student > >, 49
generateGrades< std::list< Student > >, 49
generateGrades< std::vector< Student > >, 49
generateGrades< Vector< Student > >, 49
generateNamesGrades, 50
generateNamesGrades< std::deque< Student > >, 50
generateNamesGrades< std::list< Student > >, 50
generateNamesGrades< std::vector< Student > >, 50
generateNamesGrades< Vector< Student > >, 50
handleOutput< Vector< Student > >, 50
manualInput, 50
manualInput< std::deque< Student > >, 50
manualInput< std::list< Student > >, 51
manualInput< std::vector< Student > >, 51
manualInput< Vector< Student > >, 51
readFile< Vector< Student > >, 51
scanFile, 51
scanFile< std::deque< Student > >, 51
scanFile< std::list< Student > >, 51
scanFile< std::vector< Student > >, 51

scanFile< Vector< Student > >, 52
showMeniu, 52
sortingStudents, 52
sortingStudents< std::deque< Student > >, 52
sortingStudents< std::list< Student > >, 52
sortingStudents< std::vector< Student > >, 52
sortingStudents< Vector< Student > >, 52
meniu.h
generateFile, 35
generateGrades, 35
generateNamesGrades, 35
handleOutput, 35
manualInput, 36
readFile, 36
scanFile, 36
showMeniu, 36
sortingStudents, 36

name_
Person, 23

operator<<
Student, 26
student.cpp, 56
operator>>
Student, 26
student.cpp, 56
operator=
Person, 22
Student, 25
Vector< T >, 29, 30
operator[]
Vector< T >, 30
output.cpp
handleOutput, 54
handleOutput< std::deque< Student > >, 54
handleOutput< std::list< Student > >, 54
handleOutput< std::vector< Student > >, 54
handleOutput< Vector< Student > >, 54
outputToFileSorting, 54
outputToFileSorting< std::deque< Student > >, 54
outputToFileSorting< std::list< Student > >, 54
outputToFileSorting< std::vector< Student > >, 54
outputToFileSorting< Vector< Student > >, 55
printResults, 55
printResults< std::deque< Student > >, 55
printResults< std::list< Student > >, 55
printResults< std::vector< Student > >, 55
printResults< Vector< Student > >, 55
sortContainer, 55, 56
output.h
handleOutput, 37
outputToFileSorting, 37
printResults, 38
outputToFileSorting
output.cpp, 54
output.h, 37
outputToFileSorting< std::deque< Student > >
output.cpp, 54
outputToFileSorting< std::list< Student > >
output.cpp, 54
outputToFileSorting< std::vector< Student > >
output.cpp, 54
outputToFileSorting< Vector< Student > >
output.cpp, 55

Person, 21
~Person, 21
getName, 22
getSurname, 22
name_, 23
operator=, 22
Person, 21, 22
print, 22
surname_, 23
pop_back
Vector< T >, 30
print
Person, 22
Student, 25
printResults
output.cpp, 55
output.h, 38
printResults< std::deque< Student > >
output.cpp, 55
printResults< std::list< Student > >
output.cpp, 55
printResults< std::vector< Student > >
output.cpp, 55
printResults< Vector< Student > >
output.cpp, 55
push_back
Vector< T >, 30

readFile
fileHandling.cpp, 46
fileHandling.h, 33
meniu.h, 36
readFile< std::deque< Student > >
fileHandling.cpp, 46
readFile< std::list< Student > >
fileHandling.cpp, 46
readFile< std::vector< Student > >
fileHandling.cpp, 46
readFile< Vector< Student > >
fileHandling.cpp, 46
meniu.cpp, 51
README.md, 44
reserve
Vector< T >, 30
resize
Vector< T >, 30

scanFile
meniu.cpp, 51
meniu.h, 36

scanFile< std::deque< Student > >
meniu.cpp, 51
scanFile< std::list< Student > >
meniu.cpp, 51
scanFile< std::vector< Student > >
meniu.cpp, 51
scanFile< Vector< Student > >
meniu.cpp, 52
setEgz
Student, 25
setNd
Student, 25
showMeniu
meniu.cpp, 52
meniu.h, 36
size
Vector< T >, 30
sortContainer
output.cpp, 55, 56
sortingStudents
meniu.cpp, 52
meniu.h, 36
sortingStudents< std::deque< Student > >
meniu.cpp, 52
sortingStudents< std::list< Student > >
meniu.cpp, 52
sortingStudents< std::vector< Student > >
meniu.cpp, 52
sortingStudents< Vector< Student > >
meniu.cpp, 52
src Directory Reference, 19
src/calculations.cpp, 44
src/fileHandling.cpp, 45
src/inputValid.cpp, 47
src/main.cpp, 47
src/meniu.cpp, 48
src/output.cpp, 53
src/person.cpp, 56
src/student.cpp, 56
Student, 23
~Student, 24
addNd, 25
getEgz, 25
getFinalAvg, 25
getFinalMed, 25
getNd, 25
operator<<, 26
operator>>, 26
operator=, 25
print, 25
setEgz, 25
setNd, 25
Student, 24
student.cpp
operator<<, 56
operator>>, 56
surname_
Person, 23
Vector
Vector< T >, 27
Vector< T >, 26
~Vector, 27
assign, 27
at, 27, 28
back, 28
begin, 28
capacity, 28
clear, 28
empty, 28
end, 29
erase, 29
front, 29
insert, 29
operator=, 29, 30
operator[], 30
pop_back, 30
push_back, 30
reserve, 30
resize, 30
size, 30
Vector, 27